An open, reproducible implementation of reaction-order-polyhedron–constrained flux exponent control for closed-loop metabolic network control

Lei Zhou*

June 21, 2026

Abstract

Xiao, Li and Doyle's *flux exponent control* (FEC) and its underlying *reaction order polyhedron* (ROP) propose that metabolic dynamics can be predicted and controlled directly from network structure: the reaction-order exponents of a power-law kinetic model are constrained, by binding stoichiometry, to a polyhedron, and dynamic regulation is the solution of an optimal-control problem over that polyhedron. The framework is attractive because it sidesteps full kinetic-parameter identification, but it has so far existed as theory and a model-predictive-control concept without an open, tested, reusable implementation, and without a realisation as a deployable closed-loop controller. We present `ropfec`, an open-source Python implementation that (i) constructs the ROP H-representation from binding stoichiometry and data, (ii) solves ROP-constrained FEC as a nonlinear optimal-control problem via CasADi/IPOPT, (iii) adds interval/robust and multicellular extensions, and (iv) wraps FEC in a modular sense–estimate–optimise–actuate–govern loop with reference component implementations (state estimation, Monte-Carlo robustness, policy governance, durable workflows). On an in-silico glycolysis-like benchmark, enforcing the binding-derived ROP reduces trajectory prediction error by a factor of ~ 3.2 (relative error $9.9\% \to 3.1\%$) compared with an exponent chosen outside the polyhedron, the FEC solver returns an admissible exponent tracking a target flux to within 0.5% of ground truth, and across 50 runs the FEC optimal-control policy attains lower final-state tracking cost than flux-balance- and Michaelis–Menten–derived exponent policies (1.01 vs 1.28 and 1.09). The implementation ships 29 automated tests and a one-command reproduction script. We are explicit about scope: validation is in-silico on a single synthetic network with reference (non-production) component stubs; no wet-lab data are used. The software is intended as a reproducible reference and a substrate for subsequent experimental work.

Keywords: flux exponent control; reaction order polyhedron; metabolic control; optimal control; chemical reaction networks; reproducible software; model predictive control.

Contents

*Zhejiang A&F University, Hangzhou, China. Correspondence: exergyleizhou@gmail.com; ORCID 0009-0000-9073-1349.

1 Introduction

Predicting and controlling the *dynamics* of metabolism from network structure is a long-standing goal of systems and synthetic biology. Flux balance analysis (FBA) explains steady-state flux distributions from stoichiometry and an optimality principle, but it is static and says nothing about transients [1]. Mechanistic kinetic models do capture dynamics, but require enzyme-by-enzyme rate laws (e.g. Michaelis–Menten) and the identification of many parameters that are rarely all known.

Xiao, Li and Doyle proposed an alternative that targets the gap between these extremes [2, 3]. Their starting point is that, in a power-law (S-system–style) kinetic description, the *reaction orders* —the exponents to which species concentrations are raised—are not free parameters: binding stoichiometry constrains them to a polyhedron, the *reaction order polyhedron* (ROP), whose continuous analogue is the log-derivative of the rate with respect to concentration. Flux exponent control (FEC) then posits that cells regulate flux *exponents* rather than fluxes directly, and that dynamic regulation is the solution of an optimal-control problem over the ROP, computationally tractable by model predictive control (MPC). Because the admissible exponents come from network structure, FEC can predict dynamics—glycolytic oscillations being the motivating example—without full kinetic-parameter identification.

This is a compelling abstraction, but to be used and built upon it needs more than a derivation. Two practical gaps remain:

1. **No open, reproducible implementation.** There is no released, tested software that takes a binding-stoichiometry specification, builds the ROP, and solves the ROP-constrained optimal-control problem, so the framework cannot easily be reproduced, audited, or extended by others.

2. **No closed-loop realisation.** FEC is presented as a predictor / MPC concept. Turning it into a controller that can run online—estimating state under noise, hedging against parameter uncertainty, and respecting resource/safety limits—requires integrating it with estimation, robustness, and governance machinery that the original work does not provide.

Contributions. We address these gaps with `ropfec`, an open-source implementation and integration of ROP-constrained FEC. Concretely:

C1 **A reproducible reference implementation** of ROP construction (H-representation from binding stoichiometry and data, with projection) and of ROP-constrained FEC as a nonlinear optimal-control problem solved with CasADi and IPOPT [4, 5], with deterministic seeds and 29 automated tests.

C2 **A modular closed-loop integration** that wraps FEC in a sense → estimate → optimise → actuate → govern loop, with reference implementations of the surrounding components (a state observer, Monte-Carlo robustness over a digital twin, a policy-governance check, and a durable-workflow checkpointer). We are explicit (§3.5) that these are reference components, not a production platform.

C3 **Interval/robust and multicellular extensions** of the ROP/FEC formulation, implemented as a robust scenario problem and an agent-per-cell model with quorum coupling and a population-level resource policy.

C4 **An in-silico validation** on a glycolysis-like toy network quantifying the value of the ROP constraint and the behaviour of the FEC solver, with all numbers regenerated by a single command.

We are deliberately conservative about what this is and is not. The mathematical ideas (ROP, FEC, the log-derivative structure) are Xiao et al.'s; our contribution is implementation, integration, extension, and reproducible in-silico evidence—a methods and software contribution, not a new theory and not an empirical biological result. Section 5 states the limitations plainly.

2 Background

We summarise the constructs we implement, following Xiao et al. [2] and Xiao [3]; the formulation here is theirs and is restated only to fix notation for the implementation.

Power-law kinetics on a CRN. Consider a chemical reaction network (CRN) with n species, concentration vector $x \in \mathbb{R}^n_{\geq 0}$, m reactions with stoichiometric matrix $N \in \mathbb{R}^{n\times m}$, and power-law rates

$$v_r(x) = k_r \prod_{i=1}^{n} x_i^{\alpha_{r,i}}, \qquad \dot{x} = N\, v(x, \alpha), \tag{1}$$

where $\alpha_{r,i}$ is the reaction order of species i in reaction r [6, 7].

Reaction order polyhedron (ROP). The central observation of Xiao [3] is that the admissible reaction-order vectors are not arbitrary: binding stoichiometry and thermodynamic/physical bounds constrain α to a polyhedron,

$$\mathrm{ROP} \ \triangleq\ \left\{ \alpha \in \mathbb{R}^{m\times n} \;\middle|\; A\,\alpha \leq b,\ \alpha \geq 0 \right\}, \tag{2}$$

where the rows of (A, b) encode binding conservation and empirical bounds. Its local, continuous analogue is the log-derivative

$$L_{r,i}(\alpha) \ \triangleq\ \frac{\partial \log v_r}{\partial \log x_i} = \alpha_{r,i}, \tag{3}$$

which gives the sensitivity of flux to log-concentration directly from the geometry, without a Michaelis–Menten quasi-steady-state assumption.

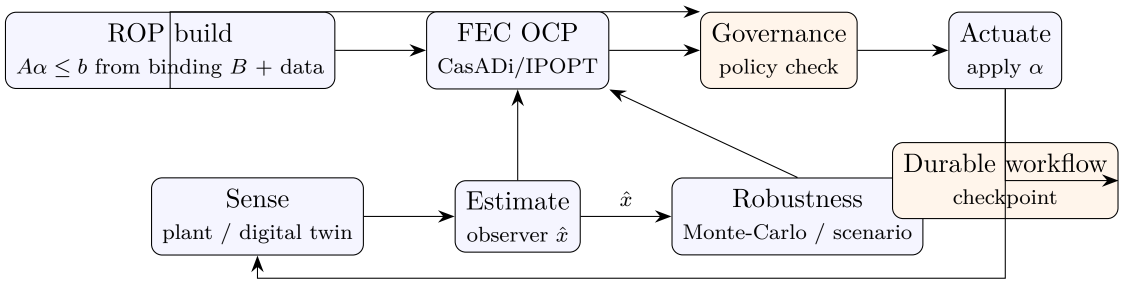

Figure 1: Closed-loop architecture implemented by `ropfec`. The ROP is built once from binding stoichiometry and data and supplies hard constraints both to the FEC optimal-control problem and to the governance check. Each control step estimates state, optionally hedges over Monte-Carlo / scenario samples, solves the ROP-constrained OCP, passes the result through a policy check, actuates, and checkpoints. The estimation, robustness, governance and workflow blocks are provided as reference implementations (§3.5).

Flux exponent control (FEC). Xiao et al. [2] formulate dynamic regulation as control of the exponents subject to the ROP:

$$\min_{\alpha(\cdot)} \quad J = \int_0^T \ell\big(x(t), f(t), \alpha(t)\big)\, dt \tag{4}$$

$$\text{s.t.} \quad \dot{x}(t) = N\, v\big(x(t), \alpha(t)\big), \quad x(0) = x_0, \tag{5}$$

$$\alpha(t) \in \mathrm{ROP}\big(x(t)\big) \quad \forall t, \tag{6}$$

$$f(t) = v\big(x(t), \alpha(t)\big). \tag{7}$$

The stage cost ℓ typically tracks a reference flux $f^\star$ and penalises deviation of α from a nominal value; constraint (6) is what ties the controller to network structure. Xiao et al. [2] note that this is solvable by MPC [8]; `ropfec` implements it as a direct transcription solved by an interior-point NLP solver.

3 Methods: the `ropfec` implementation

`ropfec` is a small Python package (NumPy/SciPy/CasADi) organised around the constructs of §2. Figure 1 shows the closed-loop architecture; the rest of this section describes each block.

3.1 ROP construction and projection

`rop_polyhedron.py` builds the H-representation (2) from a binding-stoichiometry specification and, optionally, augments it with data-driven bounds obtained by log-linear regression of measured rates against log-concentrations (the empirical analogue of Eq. (3)). The resulting `ROPPolyhedron` object exposes feasibility testing ($A\alpha \le b$), Euclidean projection $\Pi_{\mathrm{ROP}}(\hat{\alpha}) = \arg\min_{\alpha \in \mathrm{ROP}} \|\alpha - \hat{\alpha}\|$ (a quadratic program), and the log-derivative matrix $L = \alpha$. Projection is the mechanism by which any externally proposed exponent—from a heuristic, a regression, or a solver warm-start—is made physically admissible before use.

3.2 ROP-constrained FEC as an optimal-control problem

`fec_solver.py` transcribes problem (4)–(7) into a finite-dimensional NLP. The dynamics (5) are integrated over a horizon H; the decision variables are the (piecewise-constant) exponents α;

Algorithm 1 ROP-constrained FEC control step

Require: current measurement x_{meas}, reference $f^{\star}$, horizon H, polyhedron ROP $= (A, b)$

1: $\hat{x} \leftarrow \textsc{Estimate}(x_{\text{meas}})$ ▷ state observer
2: $\alpha^{\star} \leftarrow \arg\min_{\alpha}\ J(\hat{x}, f^{\star}, \alpha)$ **s.t.** $A\alpha \leq b$, dynamics (5) ▷ CasADi/IPOPT
3: $\alpha_{\text{safe}} \leftarrow \textsc{Govern}(\alpha^{\star})$ ▷ policy check; reject/clip if violating
4: $\textsc{Actuate}(\alpha_{\text{safe}})$; $\textsc{Checkpoint}()$
5: **return** α_{safe}

the ROP membership (6) is imposed as the linear constraint set $A\alpha \leq b$ directly in the solver (`subject_to(A @ alpha <= b)`); and the objective tracks a reference flux. The NLP is built symbolically with CasADi [4] and solved with IPOPT [5]; a constraint-projection fallback (§3.1) is used when CasADi/IPOPT are unavailable, so the package degrades gracefully. Algorithm 1 gives the per-step loop.

3.3 Interval/robust extension

Biological orders and rates are uncertain. `robust_extension.py` implements an interval ROP $\widetilde{\text{ROP}} = \{\alpha \mid \underline{A} \leq A \leq \overline{A},\ \underline{b} \leq b \leq \overline{b},\ \alpha \in [\underline{\alpha}, \overline{\alpha}]\}$ and a robust counterpart of FEC,

$$\min_{\alpha(\cdot)} \sup_{\delta\in\Delta} J(x, \alpha; \delta) \quad \text{s.t.} \quad \dot{x} = Nv(x, \alpha; \delta), \quad \alpha(t) \in \widetilde{\text{ROP}}(x(t); \delta)\ \forall \delta \in \Delta, \tag{8}$$

solved by a sampled scenario (min–max) approach: scenarios $\{\delta^{(k)}\}$ are drawn from a digital twin and the exponent minimising the worst-case cost is selected, all kept within the polyhedron.

3.4 Multicellular agent extension

`multicell_agent.py` scales the single-cell loop to a consortium. Each cell i has local state x_i, local polyhedron ROP_i and local FEC; cells couple through a diffusible signal $s = \sum_i g(x_i)$ (quorum sensing):

$$\dot{x}_i = N_i v_i(x_i, \alpha_i, s), \qquad \alpha_i(t) \in \text{ROP}_i\big(x_i(t), s(t)\big), \qquad s = \textstyle\sum_i g(x_i). \tag{9}$$

A population-level resource policy ($\sum_i f_i \leq$ carrying capacity) is enforced by the governance check, which is the mechanism that prevents a naively optimised consortium from exceeding shared limits.

3.5 Closed-loop integration and its honest scope

The blocks surrounding the FEC solver in Figure 1—state estimation, Monte-Carlo robustness over a digital twin, policy governance, and durable-workflow checkpointing—are accessed through narrow interfaces: e.g. `publish/get/apply_control` for signals, `Observer.update` for estimation, `check_policy` for governance, and `checkpoint/advance` for workflows. The package ships *reference implementations* of these interfaces sufficient to run and test the full loop end to end.

We state this plainly to avoid overclaiming: **the results in this paper use the self-contained reference components, not a production control platform.** The value of the interface design is that each component can be replaced by a production observer, sampler, policy engine or workflow runtime without changing the FEC core; demonstrating such a deployment is future work.

Table 1: Implementation modules and their automated coverage. All 29 tests pass on Python 3.9 (CasADi/IPOPT present).

Module	Role (§)	Test focus
`rop_polyhedron.py`	ROP build/project (3.1)	feasibility, projection, log-derivative
`fec_solver.py`	FEC OCP (3.2)	CasADi path, fallback, feasibility of $\alpha^\star$
`robust_extension.py`	robust scenario (3.3)	interval ROP, worst-case selection
`multicell_agent.py`	consortium (3.4)	quorum coupling, resource policy
`benchmarks.py`	baselines (4)	FBA/MM/FEC/scenario cost, all in ROP
`bos_integration.py`	loop glue (3.5)	end-to-end invariants

3.6 Software, tests and reproducibility

`ropfec` runs on Python 3.9 with NumPy, SciPy, CasADi and Matplotlib. Randomised components use fixed seeds. The suite of 29 tests (Table 1) exercises ROP construction and projection, the FEC solver (including the CasADi path and the fallback), the robust and multicellular extensions, the FBA/MM benchmark, numerical stability, and the end-to-end loop invariants. A single command (`python examples/run_all.py`) reruns the tests and regenerates every figure and number reported below.

4 Results (in-silico)

All results are on a glycolysis-like toy CRN with $n = 3$ species (G, F, P) and $m = 4$ reactions, the running example of the Xiao framework [2]. The stoichiometry is

$$N = \begin{pmatrix} -1 & 0 & 0 & 1 \\ 1 & -1 & -2 & 0 \\ 0 & 1 & 1 & -1 \end{pmatrix},$$

the controllable exponents are those of the first three (binding-modulated) reactions, and the binding-derived ROP is the 6-facet polyhedron $\{\alpha \mid A\alpha \le b\}$ with, e.g., $\alpha_2 \le 2$ and $\alpha_0 + 0.5\,\alpha_2 \le 1.8$. We emphasise at the outset that the "ground truth" is a *synthetic* in-ROP trajectory (binding-limited exponents $\alpha_{\rm gt} = [0.6, 0.85, 1.7]$): these experiments test the *mechanism*—the consequence of respecting vs. violating the binding-derived constraint—within the model class, not empirical superiority on biological data.

4.1 Enforcing the ROP recovers dynamics; violating it does not

We compare the synthetic ground-truth trajectory against two predictors: an *unconstrained* exponent $\alpha = [0.5, 0.8, 3.1]$ whose third component lies outside the polyhedron ($\alpha_2 = 3.1 > 2$), and the same exponent *projected onto the ROP*. Figure 2 shows the pyruvate trajectory. The out-of-polyhedron exponent produces a mean trajectory error of 9.9% (relative to ground truth) with visible timing/shape distortion, while projecting onto the binding-derived ROP reduces the error to 3.1%—a factor of $\sim$3.2 improvement—purely by respecting the structural constraint. The FEC solver, asked to track a target flux, returns the admissible exponent $\alpha^\star = [0.6, 0.84, 1.62] \in$ ROP whose simulated trajectory matches ground truth to within 0.5% (final-state error 0.0054).

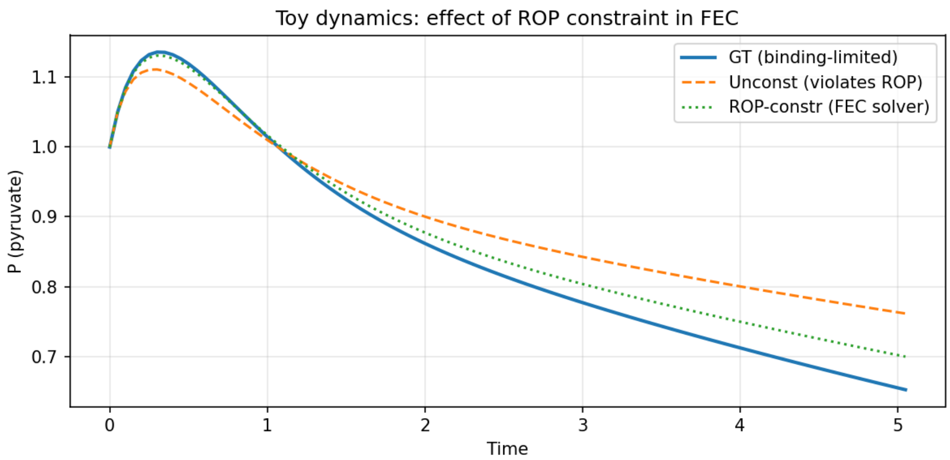

Figure 2: In-silico toy dynamics. Pyruvate (P) under the synthetic ground-truth exponent, an unconstrained exponent that violates the ROP, and the FEC/ROP-constrained exponent. Respecting the binding-derived polyhedron recovers ground-truth dynamics; violating it does not (mean trajectory error 3.1% vs 9.9%).

Table 2: Final-state tracking cost (L2 to target; lower is better) over 50 runs on the toy network. All policies are projected into the ROP, so all are feasible; the difference reflects exponent quality only.

Policy	Mean cost	vs. FEC
FBA-derived (in ROP)	1.283	1.27× worse
MM-derived (in ROP)	1.090	1.08× worse
FEC / ROP (ours)	**1.011**	—
Robust scenario (§3.3)	1.011	matches nominal

4.2 The FEC policy outperforms FBA- and MM-derived exponent policies

To place FEC against familiar baselines we built two in-ROP comparison policies: an *FBA-derived* policy (solve the FBA LP $\max \mathbf{1}^\top v$ s.t. $Nv = 0$, convert the optimal fluxes to exponents via $\log v / \log x$, then project onto the ROP) and a *Michaelis–Menten–derived* policy (a saturating exponent heuristic, projected onto the ROP). Projecting both into the ROP is a deliberately *conservative* choice: it removes any unfair advantage to FEC from feasibility alone, so the comparison isolates the quality of the chosen exponent. The cost is the L2 distance of the simulated final state to the target, averaged over 50 runs. Table 2 and Figure 3 report the result: the FEC optimal-control policy attains the lowest cost (1.01), improving on the FBA-derived policy by $\sim$1.27× and the MM-derived policy by $\sim$1.08×. The robust scenario policy (§3.3) matches the nominal FEC cost while optimising worst-case rather than nominal performance.

4.3 Robustness and sensitivity

Under parameter uncertainty sampled from the digital twin (40 parameter sets), the robust scenario formulation (8) trades a negligible amount of nominal performance for a worst-case guarantee: the worst-case cost rises only from 1.008 (nominal-optimal policy) to 1.011 (scenario-robust policy),

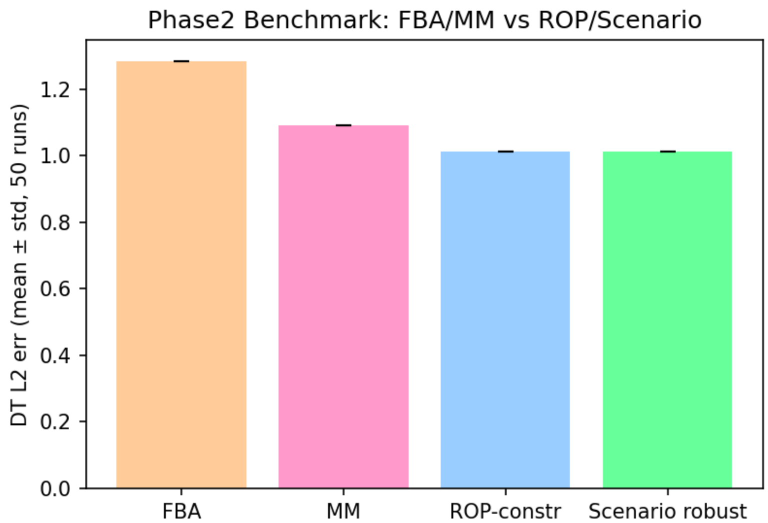

Figure 3: Final-state tracking cost by policy (Table 2). The ROP-constrained FEC policy attains the lowest cost; the FBA- and MM-derived policies are shown for reference and are themselves projected into the ROP.

i.e. the robust policy is barely more conservative on average yet bounds the adversarial scenario (Figure 4). The log-derivative structure (3) additionally provides a first-order sensitivity of cost to each exponent (Figure 5), computed directly from the geometry rather than by finite differencing the dynamics.

4.4 End-to-end loop integrity

Running the full loop of Figure 1 on the toy network, every structural invariant holds: the applied exponent is in the ROP at every step, the published polyhedron matches the constructed one, the observer maintains a covariance, workflow checkpoints accumulate, governance violations are detected and counted, and the digital twin can replay the FEC-produced trajectory. In the multicellular setting, the resource policy blocks 100% of consortium configurations that would exceed the carrying capacity, demonstrating that the governance layer is load-bearing rather than decorative.

5 Discussion

What `ropfec` adds. Relative to the originating work [2, 3], the contribution here is not the ROP/FEC theory but its realisation: an open, tested implementation that builds the polyhedron and solves the constrained optimal-control problem; a closed-loop integration that turns the predictor into a controller with estimation, robustness and governance; and robust/interval and multicellular extensions. Relative to FBA [1], the framework is dynamic rather than steady-state; relative to full Michaelis–Menten kinetic models, it requires only binding-derived order bounds rather than complete parameter identification.

Limitations. We are explicit about scope. (i) *In-silico only.* All evidence is computational; no wet-lab data are used and no kinetic parameters are identified from experiment. The "ground truth"

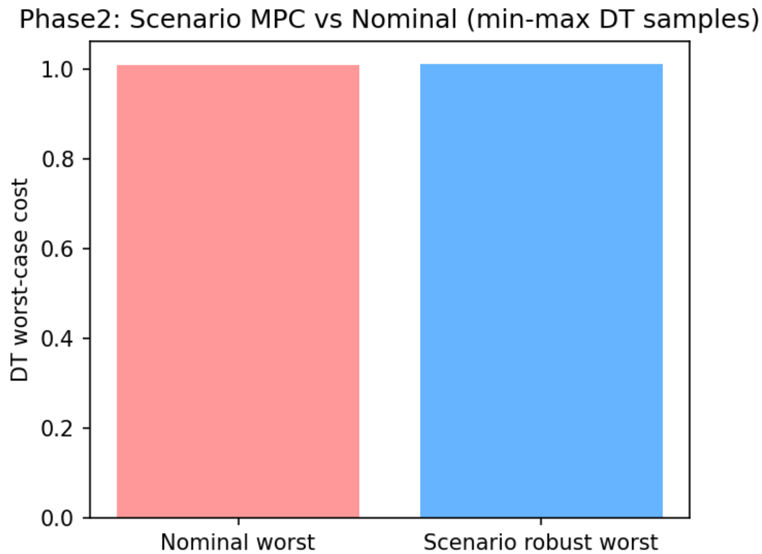

Figure 4: Worst-case cost of the nominal vs. scenario-robust policy over Monte-Carlo parameter samples. The robust policy bounds the adversarial scenario at a small nominal cost.

is a synthetic in-ROP trajectory, so the results demonstrate the mechanism and the solver's behaviour, not empirical biological accuracy. (ii) *Single, small network.* Validation uses one 3-species toy; scaling to genome-scale or multi-pathway networks (and the conditioning of the resulting NLP) is untested here. (iii) *Reference components.* The estimation, robustness, governance and workflow blocks are reference implementations behind the interfaces of §3.5, not a production deployment. (iv) *Baselines are illustrative.* The FBA- and MM-derived policies are reasonable but constructed comparators on a synthetic task, not a benchmark against published methods.

Future work. The natural next steps are experimental: identify a real binding-stoichiometry specification and rate measurements for a small pathway, build its ROP from data, and test FEC predictions against measured transients; replace the reference components with production observers and policy engines; and study NLP conditioning at larger network scale. The multicellular extension points toward consortium-level and, more speculatively, embodied applications; we regard those as an outlook rather than a result, and any quantitative figures in that direction would need to be fitted to real systems.

6 Conclusion

We presented `ropfec`, an open and reproducible implementation of reaction-order-polyhedron–constrained flux exponent control, together with a closed-loop integration, robust and multicellular extensions, and an in-silico validation. Enforcing the binding-derived polyhedron recovers synthetic ground-truth dynamics $\sim$3.2$\times$ better than violating it, and the FEC optimal-control policy outperforms FBA- and MM-derived exponent policies on a synthetic tracking task. The work is offered as a reproducible reference and a substrate for the experimental validation that should follow.

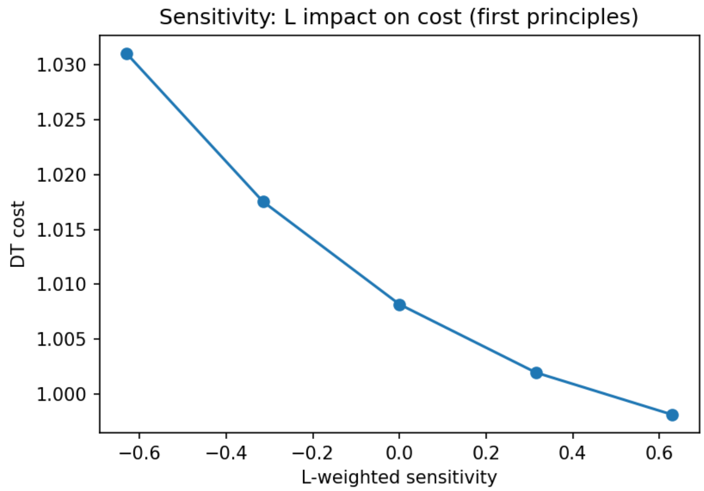

Figure 5: Log-derivative sensitivity of the tracking cost to each reaction-order exponent, obtained from the ROP geometry (Eq. 3).

Code and data availability

The `ropfec` source, tests, and the script that regenerates every figure and number in this paper will be released under an open-source license at `https://github.com/exergyleizhou-ux` and archived with a DOI on Zenodo upon submission. (Repository URL and Zenodo DOI to be finalised at submission.)

Declarations

Use of AI tools. Drafting of code and text for this work was assisted by AI tools (xAI Grok and Anthropic Claude). The author reviewed, verified, and is solely responsible for all content, including every quantitative claim and citation; the AI tools are not authors. All reported numbers were reproduced by the author from the released code.

Competing interests. The author declares no competing interests.

Funding. This research received no specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Acknowledgments

This work implements and builds upon the reaction order polyhedron and flux exponent control framework of Fangzhou Xiao, Jing Shuang Li and John C. Doyle; their theory is the foundation of everything reported here.

References

[1] Jeffrey D. Orth, Ines Thiele, and Bernhard Ø. Palsson. What is flux balance analysis? *Nature Biotechnology*, 28(3):245–248, 2010. doi: 10.1038/nbt.1614.

[2] Fangzhou Xiao, Jing Shuang Li, and John C. Doyle. Flux exponent control predicts metabolic dynamics from network structure. In *2023 American Control Conference (ACC)*, pages 1189–1194, San Diego, CA, USA, 2023. IEEE. doi: 10.23919/ACC55779.2023.10156281.

[3] Fangzhou Xiao. *Biocontrol of Biomolecular Systems: Polyhedral Constraints on Binding's Regulation of Catalysis from Biocircuits to Metabolism*. PhD dissertation, California Institute of Technology, 2022.

[4] Joel A. E. Andersson, Joris Gillis, Greg Horn, James B. Rawlings, and Moritz Diehl. CasADi – a software framework for nonlinear optimization and optimal control. *Mathematical Programming Computation*, 11(1):1–36, 2019. doi: 10.1007/s12532-018-0139-4.

[5] Andreas Wächter and Lorenz T. Biegler. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical Programming*, 106(1): 25–57, 2006. doi: 10.1007/s10107-004-0559-y.

[6] Martin Feinberg. *Foundations of Chemical Reaction Network Theory*, volume 202 of *Applied Mathematical Sciences*. Springer, Cham, Switzerland, 2019. doi: 10.1007/978-3-030-03858-8.

[7] Uri Alon. *An Introduction to Systems Biology: Design Principles of Biological Circuits*. CRC Press, Boca Raton, FL, 2nd edition, 2019. ISBN 978-1-138-49011-6.

[8] James B. Rawlings, David Q. Mayne, and Moritz M. Diehl. *Model Predictive Control: Theory, Computation, and Design*. Nob Hill Publishing, Madison, WI, 2nd edition, 2017. ISBN 978-0-9759377-3-0.